Architecture 1001:
x86-64 Assembly
Push / Pop

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

As seen in Hello.c! (on macOS)

```
_main:
0000000100003f50        pushq   %rbp
0000000100003f51        movq    %rsp, %rbp
0000000100003f54        subq    $0x10, %rsp
0000000100003f58        movl    $0x0, -0x4(%rbp)
0000000100003f5f        leaq    0x38(%rip), %rdi
0000000100003f66        movb    $0x0, %al
0000000100003f68        callq   0x100003f7e ##_printf
0000000100003f6d        movl    $0x1234, %ecx
0000000100003f72        movl    %eax, -0x8(%rbp)
0000000100003f75        movl    %ecx, %eax
0000000100003f77        addq    $0x10, %rsp
0000000100003f7b        popq    %rbp
0000000100003f7c        retq
```

macOS 11.2, Apple clang version 12.0.0 (clang-1200.0.32.29)
Disassembled from command line with “otool -tV”

As seen in Hello.c! (on Linux)

```
0000000000001149 <main>:
    1149:  endbr64
    114d:  push   %rbp
    114e:  mov    %rsp,%rbp
    1151:  lea    0xeac(%rip),%rdi
    1158:  callq  1050 <puts@plt>
    115d:  mov    $0x1234,%eax
    1162:  pop    %rbp
    1163:  retq
    1164:  nopw   %cs:0x0(%rax,%rax,1)
    116e:  xchg   %ax,%ax
```

Ubuntu 20.04, GCC 9.3.0
Disassembled with “objdump -d”


PUSH
POP
PERFECTLY BALANCED
AS ALL THINGS SHOULD BE

Is the same as…

```
0000000140001000  sub         rsp,28h
0000000140001004  lea         rcx,[__NULL_IMPORT_DESCRIPTOR+15C0h (0140006000h)]
000000014000100B  call        printf (0140001080h)
0000000140001010  mov         eax,1234h
0000000140001015  add         rsp,28h
0000000140001019  ret
```

Windows Visual Studio 2019 Community
/GS (buffer overflow protection) option turned off
Disassembled with Visual C++

Mystery Listery

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn’t?



PUSH

Push quadword onto the stack

- The push instruction automatically *decrements* the stack pointer, RSP, by 8
- *In 64-bit execution mode*, operand can be
  - The value in a 64-bit register
  - A 64-bit value from memory, as given in "r/mX" form talked about next

“r/mX” Addressing Forms 1

- “r/mX” is a term I made up to refer to anywhere you see “r/m8”, “r/m16”, “r/m32”, or “r/m64” in the Intel manual

PUSH—Push Word, Doubleword or Quadword Onto the Stack

Opcode*	Instruction	Op/ En	64-Bit Mode	Compat/ Leg Mode	Description
FF /6	PUSH *r/m16*	M	Valid	Valid	Push *r/m16.*
FF /6	PUSH *r/m32*	M	N.E.	Valid	Push *r/m32.*
FF /6	PUSH *r/m64*	M	Valid	N.E.	Push *r/m64.*
50+*rw*	PUSH *r16*	O	Valid	Valid	Push *r16.*
50+*rd*	PUSH *r32*	O	N.E.	Valid	Push *r32.*
50+*rd*	PUSH *r64*	O	Valid	N.E.	Push *r64.*
6A ib	PUSH *imm8*	I	Valid	Valid	Push *imm8.*
68 iw	PUSH *imm16*	I	Valid	Valid	Push *imm16.*
68 id	PUSH *imm32*	I	Valid	Valid	Push *imm32.*

“r/mX” Addressing Forms 2

- In Intel syntax, most of the time square brackets `[]` means to treat the value within as a memory address, and fetch the value at that address (like dereferencing a pointer)
- An “r/mX” can take 4 forms:
  1. Register → `rbx`
  2. Memory, base-only → `[rbx]`
  3. Memory, base+index*scale → `[rbx+rcx*X]`
     - For X = 1, 2, 4, or 8
  4. Memory, base+index*scale+displacement → `[rbx+rcx*X+Y]`
     - For Y of 1 byte (0-2^8) or 4 bytes (0-2^32)

“r/mX” Addressing Forms 3

`[base + index*scale + displacement]`

- Has natural applicability to multi-dimensional array indexing, arrays of structs, etc

- So when I say in the future instructions support access to memory, I mean memory as encoded in an “r/mX” form
- And when I say “r/mX”, I mean something that could be as simple as a single register, or as complicated as a memory address calculation in that form

“r/mX” Addressing Example

push register
`push rbx`
push memory
`push [rbx]`
`push [rbx+rcx*4]`
`push [rbx+rcx*8+0x20]`

Note: there’s nothing special about the particular registers I chose above. Any could be substituted with any other general purpose registers

Note about the ` address convention

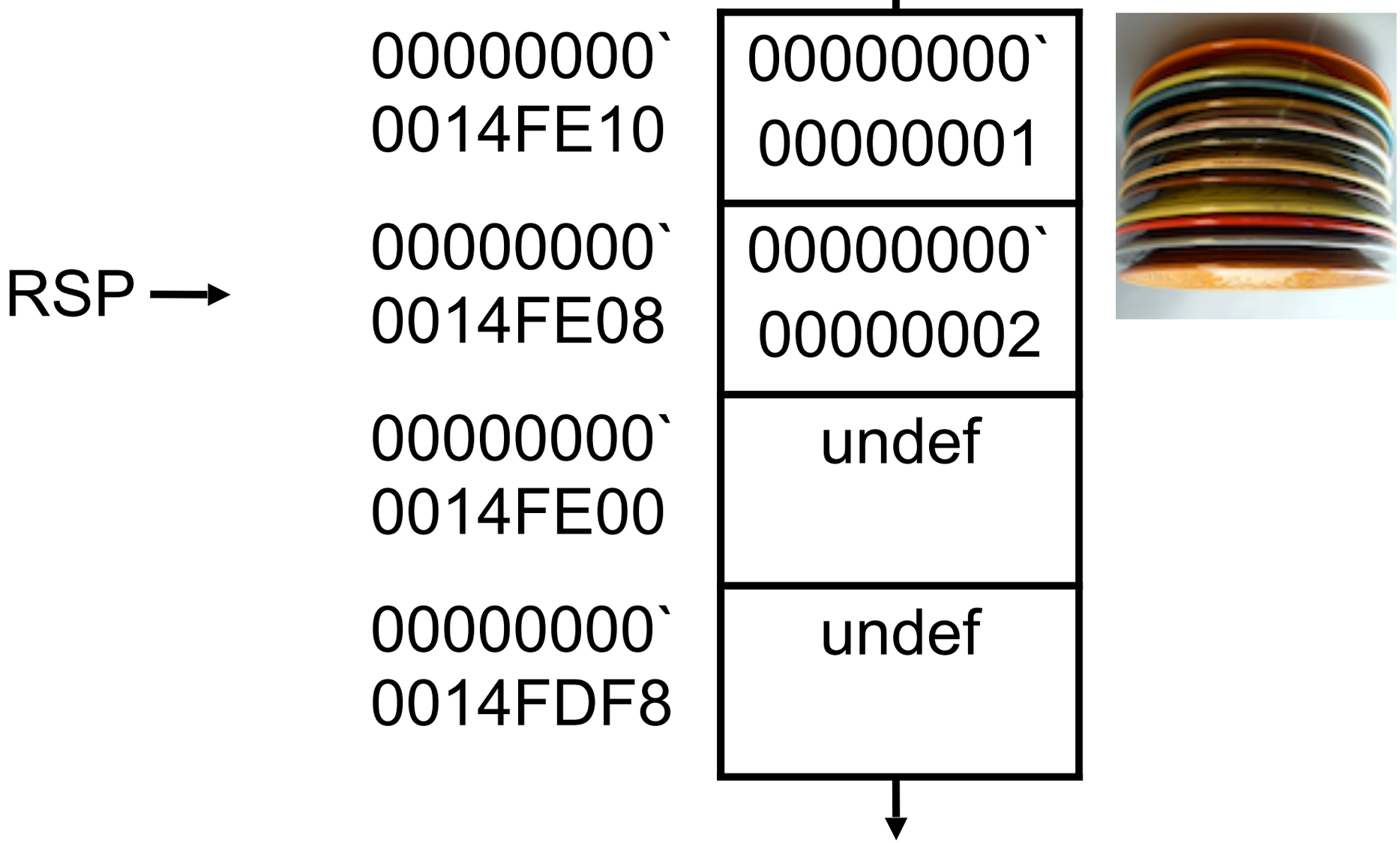
- When writing 64 bit numbers, it can be easy to lose track of whether you have the right number of digits
- WinDbg (which we don't use in this class, but do in future architecture classes) allows you to write 64 bit numbers with a ` between the two 32 bit halves.
- I think this is helpful to see when a number is > 32 bit or not (because there will be some non-zero value on the left side of the `)
- So in this class I'll often write 64 bit numbers like 0x12345678`12345678.
- But keep in mind that the only tool which probably supports you entering them like that is WinDbg

push RAX

Registers Before

RAX	00000000` 00000003
RSP	00000000` 0014FE08

Stack Before



push RAX

Registers Before

RAX	00000000` 00000003
RSP	00000000` 0014FE08

Registers After

RAX	00000000` 00000003
RSP	00000000` 0014FE00

Stack Before

RSP →

00000000`0014FE10	00000000`00000001
00000000`0014FE08	00000000`00000002
00000000`0014FE00	undef
00000000`0014FDF8	undef



Stack After

RSP →

00000000`00000001	
00000000`00000002	
00000000`00000003	
undef	



POP
Pop a value from the stack

- *In 64-bit execution mode*, operand can be
  - A 64-bit register
  - A 64-bit memory address, as given in "r/mX" form
- Increments RSP by 0x8

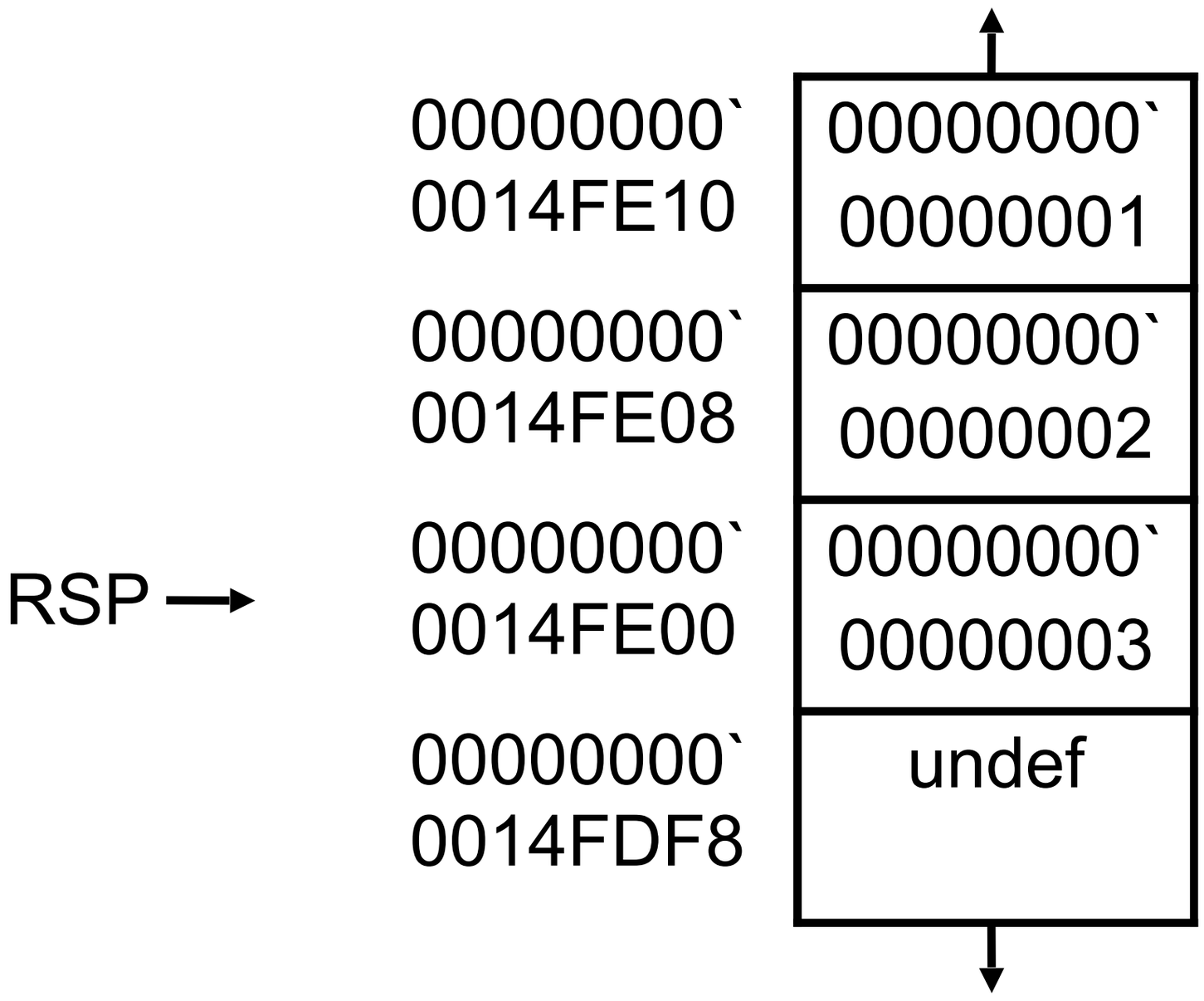
Book page 108

pop RAX

Registers Before

RAX	00000000` 00005007
RSP	00000000` 0014FE00

Stack Before



pop RAX

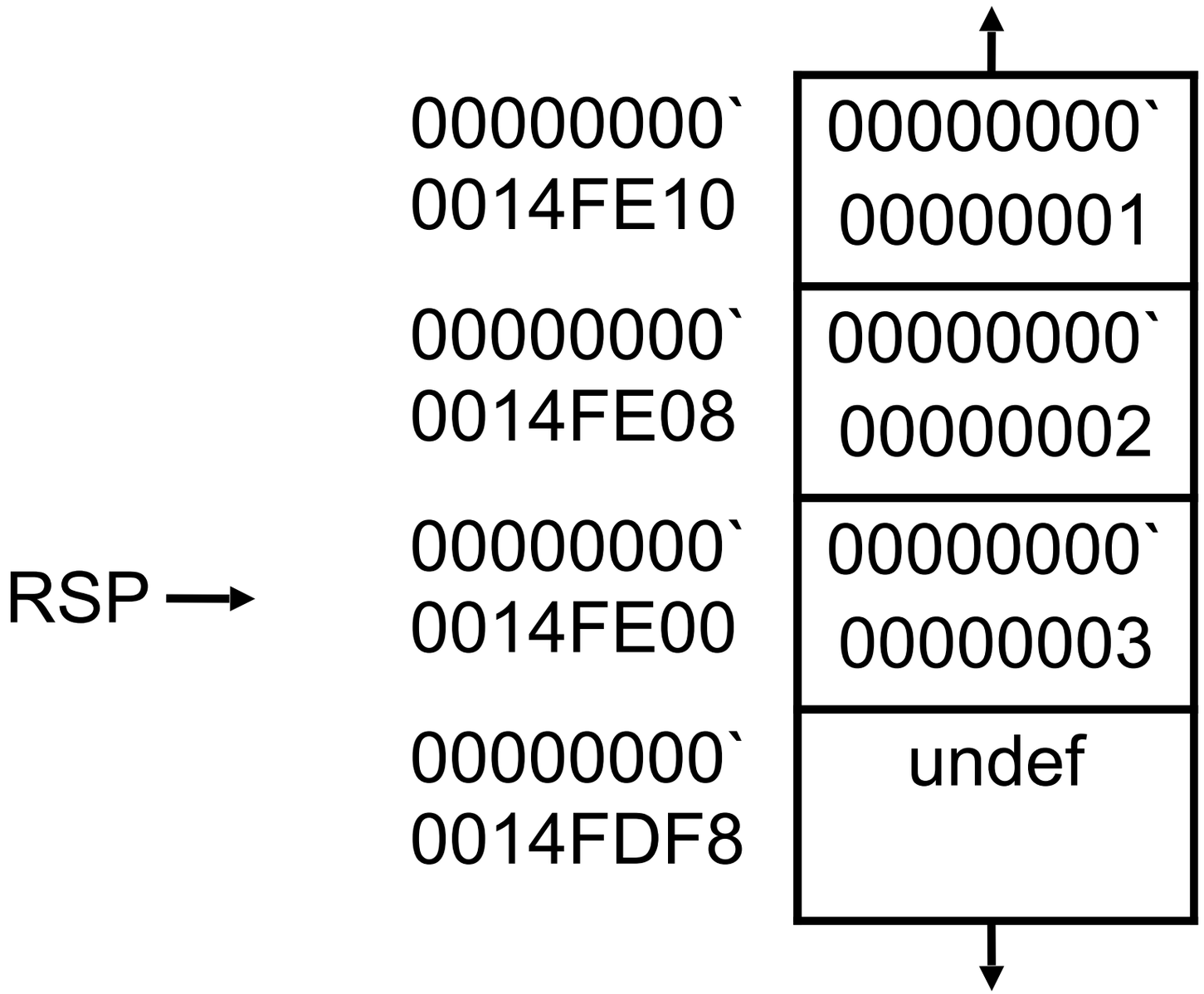
Registers Before

RAX	00000000` 00005007
RSP	00000000` 0014FE00

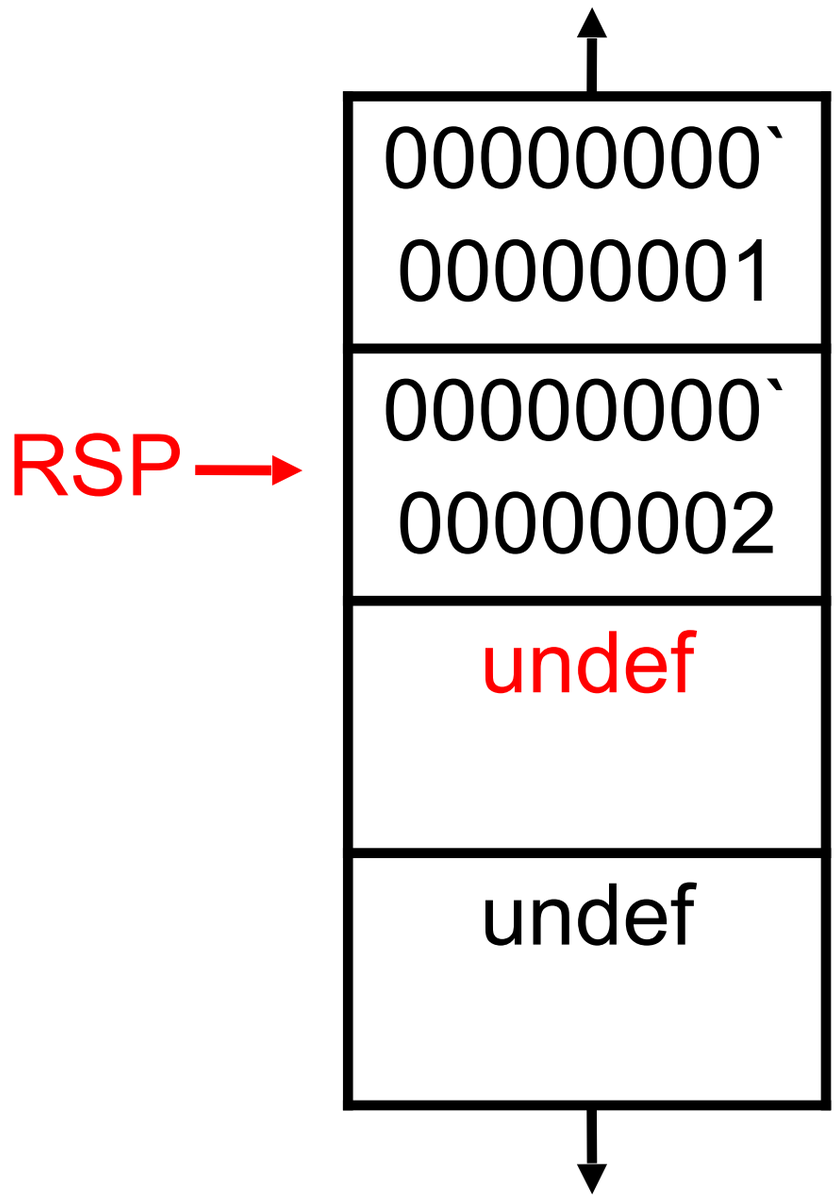
Registers After

RAX	00000000` 00000003
RSP	00000000` 0014FE08

Stack Before



Stack After



Push/Pop 32-bit throwback

- If you are executing in 32-bit mode, push/pop will add/remove values 32 bits at a time, rather than 64 bits, and thus they decrement/increment *ESP* by 4 rather than 8 at a time
- Likewise, if you're in 16-bit mode, they push/pop 16-bit values, and decrement/increment *SP* by 2 at a time

Instructions we now know (3)

- NOP
- **PUSH/POP**

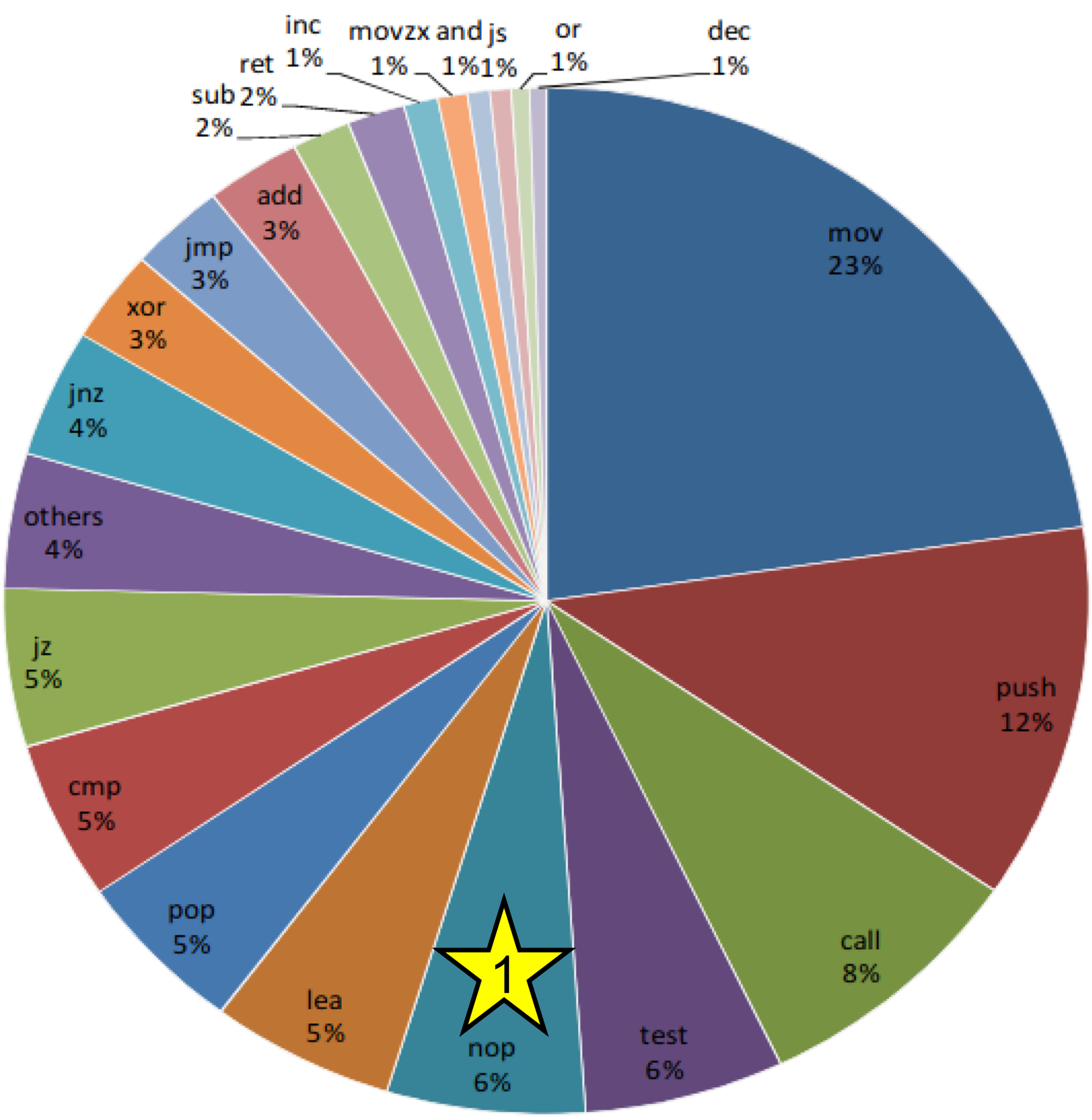


Fig. 2 Web browsers' instruction frequencies

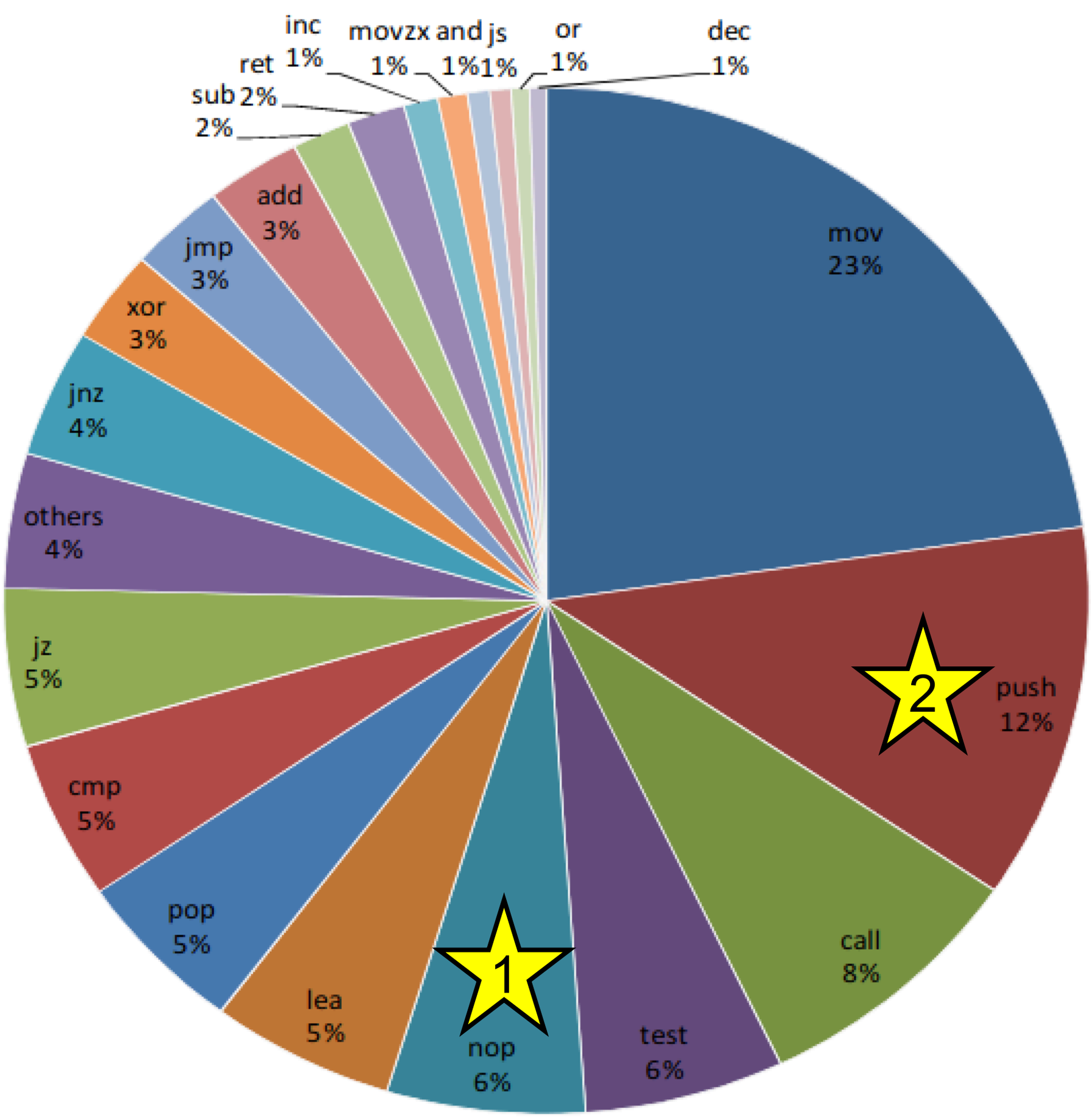


Fig. 2 Web browsers' instruction frequencies

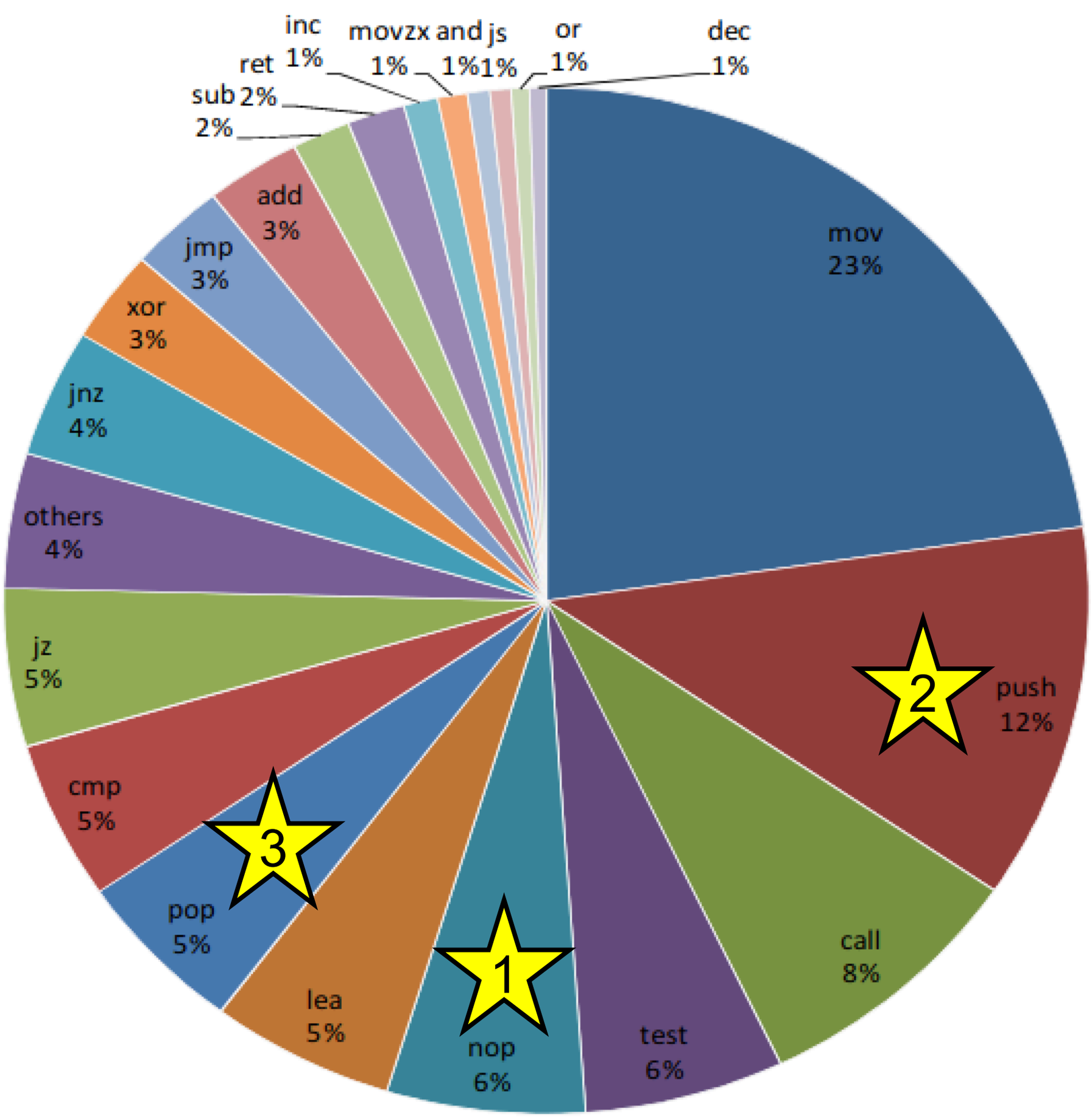


Fig. 2 Web browsers' instruction frequencies